

Práctica 3: RMI

Desarrollo de Sistemas Distribuidos

1. Introducción:

En esta práctica nos vamos a meter de lleno en el mundo de los sistemas distribuidos utilizando **Java RMI** (Remote Method Invocation). El objetivo principal es comprender cómo dos programas que se están ejecutando en máquinas totalmente distintas pueden comunicarse entre sí. Al final de la práctica aplicaremos estos conocimientos para construir un sistema más complejo, **un servidor para gestionar donaciones**.

Para entender lo que estamos haciendo, vamos a repasar brevemente los conceptos teóricos clave sobre los que se apoya nuestro código:

- **EL objetivo de RMI:** Permite que un cliente ejecute funciones de un servidor remoto como si fuese local, ocultando toda la complejidad de la red.
- **Las interfaces:** Definen qué operaciones están disponibles para el cliente sin revelar cómo están programadas por dentro.
- **Los Stubs:** Son objetos locales que actúan como intermediarios. Recogen la petición del cliente, la envían al servidor real por la red y le devuelven la respuesta si es que la hubieran.
- **Paso por copia:** Los objetos locales se envían haciendo una copia exacta de ellos, mientras que los objetos remotos se pasan siempre por referencia.

2. Entorno de Desarrollo y Despliegue

Aunque las instrucciones originales del guión plantean el desarrollo utilizando el IDE de programación NetBeans, para la realización de esta práctica se ha optado por utilizar **Visual Studio Code (VS Code)**.

El motivo principal es la agilidad. NetBeans por su forma de gestionar proyectos nos obligaría a crear un proyecto distinto para la interfaz, otro para el

cliente y otro para el servidor, teniendo que empaquetar manualmente las interfaces en archivos.jar para importarlas de un lado a otro cada vez que hagamos un cambio.

Con VS Code y la extensión de Java, podemos trabajar con todo en un único espacio de trabajo organizado por paquetes dentro de la misma carpeta fuente src. El propio editor se encarga de resolver las dependencias entre el cliente y el servidor de forma transparente.

Es por ello que una estructura del proyecto quedaría organizada de la siguiente manera:

```
Practica3_RMI/
├── src/
│   ├── iconrador/
│   │   └── iconrador.java    (La interfaz remota)
│   ├── contador/
│   │   └── contador.java    (La implementación)
│   ├── servidor/
│   │   ├── servidor.java    (El lanzador del servidor)
│   │   └── server.policy    (Políticas de seguridad del servidor)
│   └── cliente/
│       ├── cliente.java     (El programa cliente)
│       └── client.policy    (Políticas de seguridad del cliente)
```

Compilación y ejecución:

Para ejecutar los distintos ejemplos a lo largo del desarrollo he aprovechado la terminal integrada de VS Code. Esto nos permite simular un entorno de producción real utilizando directamente las herramientas del sistema (como javac para compilar y java para lanzar las aplicaciones).

Por ejemplo, el flujo de trabajo normal para arrancar nuestros sistemas consta de los siguientes pasos desde la terminal:

1. **Compilación global:** “javac *.java”
2. **Levantar el registro de RMI:** “rmiregistry &”
3. **Lanzar el servidor:** “java -cp. -Djava.security.policy=server.policy servidor.servidor &”
4. **Lanzar los clientes:**
Se conectan al registro para buscar al servidor y empezar a interactuar.

3. Implementación y análisis de los ejemplos

En esta primera fase del proyecto, he implementado y analizado los tres ejemplos que se proponen en el guión para comprender el comportamiento interno de RMI antes de enfrentarnos al desarrollo del servidor replicado. Vamos a ir desempolvando cada uno de los ejemplos para comprender su funcionamiento:

- 3.1 Primer ejemplo:

En este primer ejemplo se establece una arquitectura básica de un sistema en RMI. Para ponerlo en marcha hemos utilizado los 4 ficheros fundamentales que se definían en el guión:

- **Ejemplo_I.java (La interfaz):** Aquí establecemos el contrato entre el cliente y el servidor. Definimos la firma del método *escribir_mensaje* indicando que puede ser invocado de forma remota y que puede lanzar una excepción.
- **Ejemplo.java (El servidor):** Basicamente es el programa que implementa la interfaz anterior. Contiene la lógica real del método *escribir_mensaje*. Donde para comprobar la concurrencia, tenemos que si el cliente le pasa un 0, el servidor se queda dormido durante 5 segundos. Además este fichero contiene el main que levanta el servidor.
- **Cliente_Ejemplo.java (El cliente):** Es el programa que busca el servidor en el registro del RMI local, obtenemos su stub y ejecutamos la llamada remota pasándole el número que le indiquemos por consola.
- **server.policy (Políticas de seguridad):** Un fichero de configuración que le da permisos a Java para que acepte las conexiones de red y tal.

Ejecución y comandos:

Para ver perfectamente el ciclo de ejecución del programa sin que los mensajes se solaparan en la consola, he dividido mi entorno de VS Code en **tres terminales distintas**. En cada una de ellas he ejecutado lo siguiente:

TERMINAL 1: "rmiregistry"

Este comando simplemente levanta el servicio, se queda ejecutándose en segundo plano.

TERMINAL 2: "java -Djava.security.manager=allow -cp -Djava.security.policy=server.policy Ejemplo"

Aquí ejecutamos el servidor. Aclarar que con el “-cp .” le decimos que busque las clases en el directorio actual, y con “-Djava.security.policy=server.policy” cargamos el fichero de permisos.

TERMINAL 3: “java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy Cliente_Ejemplo localhost 0 & java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy Cliente_Ejemplo localhost 3”

Aquí lanzamos los clientes casi al mismo tiempo lanzando el primer cliente y enviándolo al segundo plano con &.

Además hay que aclarar que al ejecutar los comandos, la consola muestra varios avisos indicando Warnings de seguridad. Esto se debe a que el código hace uso de funciones que se han **quedado obsoletas** en versiones modernas de Java. Por ese motivo he tenido que añadir en los comandos el flag de “-Djava.security.manager=allow” para que nos permita utilizarla y funcione todo correctamente.

El objetivo principal de este ejemplo es observar cómo se comporta el servidor al recibir la segunda petición mientras que la primera sigue durmiendo. A continuación se muestra la captura de la tres terminales durante la ejecución:

```

zapi24@zapi24-GF63-Thin-9SC:~/Escritorio
o/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo1$ rmiregistry
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release
[]

o/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo1$ java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy Cliente_Ejemplo localhost 0 &
java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy Cliente_Ejemplo localhost 3
[1] 9004
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by Cliente_Ejemplo (file:/home/zapi24/Escritorio/3º202%2baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo1/)
WARNING: Please consider reporting this to the maintainers of Cliente_Ejemplo
WARNING: System::setSecurityManager will be removed in a future release
Buscando el objeto remoto
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by Cliente_Ejemplo (file:/home/zapi24/Escritorio/3º202%2baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo1/)
WARNING: Please consider reporting this to the maintainers of Cliente_Ejemplo
WARNING: System::setSecurityManager will be removed in a future release
Buscando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto

o/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo1$
zapi24@zapi24-GF63-Thin-9SC:~/Escritorio
o/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo1$ java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy Ejemplo
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by Ejemplo (file:/home/zapi24/Escritorio/3º202%2baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo1/)
WARNING: Please consider reporting this to the maintainers of Ejemplo
WARNING: System::setSecurityManager will be removed in a future release
Ejemplo bound
Recibida peticion de proceso: 0
Empezamos a dormir
Recibida peticion de proceso: 3

Hebra 3
Terminamos de dormir

Hebra 0
[]
  
```

Si nos fijamos en la salida de la segunda terminal que es la importante, podemos comprobar la gestión del multihilo de RMI:

1. El servidor recibe la petición del proceso 0.
2. Imprime “Empezamos a dormir” y su hilo de ejecución se congela 5 segundos.
3. Acepta inmediatamente después la petición del proceso 3, atendiendo a ese cliente e imprimiendo por pantalla.
4. Finalmente tras los 5 segundos, el hilo original despierta y termina imprimiendo “Hebra 0”

Para cada llamada remota entrante, el servidor RMI asigna un thread independiente, permitiendo que múltiples peticiones de distintos clientes entren de forma simultánea sin generar cuellos de botella.

- 3.2 Segundo ejemplo:

En este segundo ejemplo vamos a profundizar en la gestión de la concurrencia de RMI. En lugar de simular múltiples clientes abriendo varias terminales, se ha modificado el sistema para que un único programa cliente sea capaz de generar múltiples peticiones utilizando los threads. Para ello hemos reciclado las políticas de seguridad del ejemplo 1 y añadido los siguientes cambios:

- **Ejemplo_I.java y Ejemplo.java:** Se ha modificado la interfaz para que ahora el método *escribir_mensaje* ahora reciba un string, además de hacer que la salida sea más intuitiva con mensajes que muestran cada vez que entra y sale una hebra.
- **Cliente_Ejemplo_Multi_Threaded.java:** Este nuevo cliente implementa la interfaz Runnable. Cuando lo ejecutamos le pasamos por argumento el número de clientes que queremos simular, que en nuestro caso va a ser 5. El programa se encarga de crear un array de hebras y las lanza todas de golpe contra el servidor.

Para que se vea su funcionamiento correctamente, se ha realizado dos ejecuciones distintas:

PRIMERA EJECUCIÓN:

Hemos lanzado el servidor y ejecutado el cliente utilizando 5 hebras simultáneas. A continuación se inserta la captura de dicha ejecución:

```

2°Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$
zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2°Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$ java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy Ejemplo
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by Ejemplo (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2/)
WARNING: Please consider reporting this to the maintainers of Ejemplo
WARNING: System::setSecurityManager will be removed in a future release
Ejemplo bound

Entra Hebra Cliente 4

Entra Hebra Cliente 1
Sale Hebra Cliente 1

Entra Hebra Cliente 3
Sale Hebra Cliente 3

Entra Hebra Cliente 0
Empezamos a dormir

Entra Hebra Cliente 2
Sale Hebra Cliente 2
Sale Hebra Cliente 4
Terminamos de dormir
Sale Hebra Cliente 0
^Czapi24@zapi24-GF63-Thin-9SC:~/Escritorio/

2°Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$
zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2°Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by Cliente_Ejemplo_Multi_Threaded (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2/)
WARNING: Please consider reporting this to the maintainers of Cliente_Ejemplo_Multi_Threaded
WARNING: System::setSecurityManager will be removed in a future release
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2°Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$

```

Analizando la salida, vemos el entrelazado de mensajes. Las peticiones llegan al servidor el cual las ejecuta en paralelo. El momento importante ocurre cuando el cliente 0 imprime el “Empezamos a dormir” y ese thread espera durante 5 segundos. Durante ese tiempo el servidor acepta al cliente 2 completamente, y termina de procesar al 4. Solo al finalizar la pausa, el cliente 0 despierta y termina su ejecución. Esto nos explica que los objetos remotos en RMI deben ser programados de forma segura en concurrencia, ya que sus métodos pueden ser invocados por múltiples hilos simultáneamente.

SEGUNDA EJECUCIÓN:

Aquí para evitar los problemas de condiciones de carreras que puedan ocurrir por el acceso concurrente, el guión propone añadir el modificador `synchronized`. Así que lo añadimos y al recompilar y volver a ejecutar el programa para 5 hebras obtenemos la siguiente salida:

```

2°Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$
zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2°Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release
[]

3
2°Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$
zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2°Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by Ejemplo (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2/)
WARNING: Please consider reporting this to the maintainers of Ejemplo
WARNING: System::setSecurityManager will be removed in a future release
Ejemplo bound

Entra Hebra Cliente 3
Sale Hebra Cliente 3

Entra Hebra Cliente 4
Sale Hebra Cliente 4

Entra Hebra Cliente 0
Empezamos a dormir
Terminamos de dormir
Sale Hebra Cliente 0

Entra Hebra Cliente 1
Sale Hebra Cliente 1

Entra Hebra Cliente 2
Sale Hebra Cliente 2

4Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$
zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 4Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by Cliente_Ejemplo_Multi_Threaded (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2/)
WARNING: Please consider reporting this to the maintainers of Cliente_Ejemplo_Multi_Threaded
WARNING: System::setSecurityManager will be removed in a future release
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 4Cuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo2$

```

Como podemos ver, el comportamiento del programa ha cambiado. El `synchronized` fuerza a Java a tratar el método como una sección crítica, por lo que solo un hilo puede ejecutarlo al mismo tiempo.

Si vemos la consola podemos apreciar que desaparece completamente el entrelazado, los clientes entran y salen secuencialmente. Cuando el cliente 0 entra y comienza a dormir, no deja pasar al resto de threads bloqueándolas. No es hasta que termina que libera el método y deja entrar al resto.

Aunque esto soluciona los problemas de concurrencia, se nos queda un poco limitado, ya que el `synchronized` es insuficiente para sincronizar variables distribuidas entre distintas réplicas de servidores.

- 3.3 Tercer ejemplo:

En este último ejemplo dejamos de lado la gestión de hebras y nos enfocamos en por un lado la simplificación del despliegue y por otro en una medición de la latencia que provocan nuestras llamadas a través de la red. Para montar esta arquitectura hemos creado un sistema de contador utilizando los siguientes archivos (cabe recalcar que el `server.policy` lo seguimos reutilizando de los ejemplos anteriores):

- **icontador.java y contador.java:** Aquí se implementa un objeto remoto que almacena un valor entero para sumarlo e incrementarlo. Tenemos que heredar de *UnicastRemoteObject*.
- **servidor.java:** La chicha de este archivo está en la línea *LocateRegistry.createRegistry(1099)*. Con esta instrucción el código levanta internamente el servicio RMI para que no tengamos que hacerlo nosotros a mano a través de la terminal.
- **cliente.java:** Se conecta al servidor inicializando el contador a 0, y se encarga de lanzar un bucle que invoca la función remota *incrementar()* 1000 veces consecutivas, calculando el tiempo medio que tarda en completarse cada llamada.

Ejecución y comandos:

Como hemos automatizado el registro RMI dentro del código del servidor, ahora solo necesitaremos dos terminales:

TERMINAL 1 (Servidor): Simplemente después de compilar el código con “javac *.java” lanzamos el servidor con: “java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy servidor”

TERMINAL 2 (Cliente): Ejecutamos el cliente para iniciar la ráfaga de 1000 llamada con: “java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy cliente”. A continuación muestro la salida tras finalizar la ejecución:

```

zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo3$ java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy servidor
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by servidor (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo3/)
WARNING: Please consider reporting this to the maintainers of servidor
WARNING: System::setSecurityManager will be removed in a future release
Servidor RemoteException | MalformedURLExceptionor preparado
█

zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo3$ java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy cliente
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by cliente (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo3/)
WARNING: Please consider reporting this to the maintainers of cliente
WARNING: System::setSecurityManager will be removed in a future release
Poniendo contador a 0
Incrementando...
Media de las RMI realizadas = 0.131 msecs
RMI realizadas = 1000
zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Ejemplo3$ █

```

El cliente nos confirma que se han realizado las 1000 invocaciones remotas. Mostrándonos que el tiempo medio por RMI realizada es de **0.131 milisegundos**. Este tiempo tan reducido se debe a que tanto el cliente como el servidor se están ejecutando obviamente en la misma máquina física, por lo que no hay ninguna latencia de red. Aun así se demuestra que aunque para el programador se vea como

invocar una función local, por debajo hay implementado un proceso de empaquetado, serialización y envío (de lo que se encarga el stub) que siempre añade un pequeño coste de overhead a la ejecución.

4. Servidor Replicado de Donaciones

En esta parte de la práctica hemos aplicado todo lo que hemos aprendido durante la síntesis de los ejemplos del guión para desarrollar una plataforma de donaciones distribuida y replicada. Para optar a la máxima calificación posible, he diseñado el sistema de forma escalable, es decir, aunque en la entrega por simplicidad solo se propongan dos réplicas, podríamos implementar las que quisiéramos.

Para ello se ha tenido en cuenta que sea seguro dentro de la concurrencia, he utilizado un *ConcurrentHashMap* además de incluir obviamente roda la interacción por consola de forma dinámica para el cliente.

- Arquitectura del sistema:

El sistema de archivos de este sistema obtenido a través del comando tree de la terminal tiene la siguiente pinta:

```
Practica3_Donaciones/
├── src/
│   ├── interfaces/
│   │   └── IDonaciones.java           (El contrato entre clientes y réplicas)
│   ├── servidor/
│   │   ├── ServidorDonaciones.java   (La lógica: registrar, donar, balancear...)
│   │   ├── LanzadorServidor.java     (El main que levanta una réplica)
│   │   └── server.policy              (Políticas de seguridad)
│   └── cliente/
│       ├── Cliente.java              (El menú interactivo para el usuario)
│       └── client.policy              (Políticas de seguridad)
```

Dentro de estos archivos, vamos a destacar y explicar brevemente los más importantes para el funcionamiento del código. Mencionando también que se han intentado añadir muchos comentarios dentro del código mismo para que este sea lo más legible y fácil de entender.

- **interfaces.IDonaciones:** Se encarga de definir el contrato. Contiene los métodos públicos del cliente de registrar, donar y consultar. Además de todos los métodos internos para que las réplicas se comuniquen entre sí, como los de pasarse subtotales, consultar el número de clientes etc
- **servidor.ServidorDonaciones y LanzadorServidor:** Estos archivos contienen la lógica de negocio. El lanzador permite instanciar múltiples réplicas pasándoles por parámetro su puerto y la URL de sus compañeras.
- **cliente.Cliente:** Aquí se define el menú interactivo con el que el usuario interactúa, y donde se procesan las peticiones con el stub las conexiones en tiempo real con el servidor.

- **Ejecución y pruebas del sistema:**

Para la ejecución del programa, es muy importante abrir **tres terminales independientes** para ver el flujo completo de este. En las dos primeras hemos levantado las réplicas pasándoles la referencia cruzada de la otra, y en la tercera hemos ejecutado el cliente:

TERMINAL 1:

```
"java -Djava.security.manager=allow -cp .  
-Djava.security.policy=servidor/server.policy servidor.LanzadorServidor 1099  
Replica1 rmi://localhost:1100/Replica2"
```

TERMINAL 2:

```
"java -Djava.security.manager=allow -cp .  
-Djava.security.policy=servidor/server.policy servidor.LanzadorServidor 1100  
Replica2 rmi://localhost:1099/Replica1"
```

TERMINAL 3:

```
"java -Djava.security.manager=allow -cp .  
-Djava.security.policy=cliente/client.policy cliente.Cliente"
```

A continuación vamos a mostrar dos escenarios distintos de ejecución para demostrar el cumplimiento de los requisitos plasmados en el guión, además del funcionamiento correcto del código:

- 1º Primera prueba:

En esta prueba se plasma como las réplicas se comunican entre sí, alternándose para balancear la carga de usuarios además que ver cómo el cliente se adapta sin que el usuario no sufra ninguna interrupción. El hilo de ejecución que se ha seguido es el siguiente:

1. Iniciamos el cliente y solicitamos conectarnos a la Replica1 en el puerto 1099.
2. Seleccionamos la opción 1 (registrarse) e introducimos el nombre de Zapi.
3. Volvemos a seleccionar la opción 1 e introducimos el nombre de Paco.

```

zapi24@zapi24-GF63-Thin-95C:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src$ java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy servidor.LanzadorServidor 1099 Replic1 rmi://localhost:1100/Replica2
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by servidor.LanzadorServidor (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src/)
WARNING: Please consider reporting this to the maintainers of servidor.LanzadorServidor
WARNING: System::setSecurityManager will be removed in a future release

=====
Replica1 ACTIVA y escuchando en puerto 1099
Conectada a 1 replicas compañeras.
=====
Petición de registro recibida de: Zapi
Cliente Zapi registrado internamente.
Petición de registro recibida de: Paco
Cliente Paco redirigido a rmi://localhost:1100/Replica2
[]

zapi24@zapi24-GF63-Thin-95C:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src$ java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy servidor.LanzadorServidor 1100 Replica2 rmi://localhost:1099/Replica1
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by servidor.LanzadorServidor (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src/)
WARNING: Please consider reporting this to the maintainers of servidor.LanzadorServidor
WARNING: System::setSecurityManager will be removed in a future release

=====
Replica2 ACTIVA y escuchando en puerto 1100
Conectada a 1 replicas compañeras.
=====
Cliente Paco registrado internamente.
[]

zapi24@zapi24-GF63-Thin-95C:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src$ java -Djava.security.manager=allow -cp . -Djava.security.policy=cliente/cliente.policy cliente.Cliente
Introduce el puerto del registro RMI (ej. 1099): 1099
Introduce el nombre de la replica a la que conectar (ej. Replic1): Replic1

[OK] Conectado exitosamente a Replic1

--- MENU DE OPCIONES ---
Conectado actualmente a: Replic1
1. Registrarse como nuevo donante
2. Realizar una donacion
3. Consultar total recaudado (Global)
4. Ver lista de donantes (Global)
5. Salir
Elige una opcion: 1
Introduce tu nombre o ID para registrarte: Zapi
>> Registro completado.

--- MENU DE OPCIONES ---
Conectado actualmente a: Replic1
1. Registrarse como nuevo donante
2. Realizar una donacion
3. Consultar total recaudado (Global)
4. Ver lista de donantes (Global)
5. Salir
Elige una opcion: 1
Introduce tu nombre o ID para registrarte: Paco
>> Registro completado.
>> INFO: Por balanceo de carga, el servidor te ha redirigido.
>> Redirigiendo conexion de forma transparente...

--- MENU DE OPCIONES ---
Conectado actualmente a: Replica2
1. Registrarse como nuevo donante
2. Realizar una donacion
3. Consultar total recaudado (Global)
4. Ver lista de donantes (Global)
5. Salir
Elige una opcion:

```

En primer lugar, la salida de la consola del cliente confirma que la conexión inicial con la Réplica 1 se realiza correctamente, cuando se nos muestra por pantalla el mensaje de *[OK] Conectado exitosamente a Replic1*.

Al registrar al primer usuario Zapi, como ambas réplicas tienen cero clientes, la Replica1 asume la carga y lo registra internamente.

A su vez, cuando intentamos registrar a Paco de nuevo en la Replica1. EL servidor detecta mediante una llamada interna que su compañera Replica2 estás más vacía. En ese momento, la primera réplica rechaza quedarse con Paco y lo registra directamente en la Replica2 devolviendo la URL de la nueva réplica. Nuestro programa cliente lee esta URL, actualizando su conexión internamente y el menú

muestra por pantalla el mensaje de “Conectado actualmente a: Replica2”, garantizando así la transparencia.

- 2º Segunda prueba:

En esta prueba lo que vamos a demostrar es que aunque los clientes y el dinero estén repartidos en distintos servidores físicos, el sistema funciona de forma coherente como un todo. El hilo de ejecución que se ha seguido es el siguiente:

1. Estando conectamos a la Replica2 (tras el balanceo anterior) seleccionamos la opción 2 (Donar) y donamos 100 euros con el ID de Paco.
2. Cerramos el cliente (Opción 5), lo volvemos a ejecutar y esta vez nos conectamos a la Replica1 manualmente.
3. Seleccionamos la opción 2 y donamos 50 euros con el ID de Zapi.
4. Seleccionamos la opción 3 para consultar el total global.
5. Seleccionamos la opción 4 para ver la lista global de donantes.

```

zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src$ java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy servidor.LanzadorServidor 1099 Replica1 rmi://localhost:1100/Replica2
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by servidor.LanzadorServidor (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src/)
WARNING: Please consider reporting this to the maintainers of servidor.LanzadorServidor
WARNING: System::setSecurityManager will be removed in a future release

=====
Replica1 ACTIVA y escuchando en puerto 1099
Conectada a 1 replicas compañeras.
=====
Petición de registro recibida de: Zapi
Cliente Zapi registrado internamente.
Petición de registro recibida de: Paco
Cliente Paco redirigido a rmi://localhost:1100/Replica2
Donación de 50.0 recibida de Zapi

```

```

zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src$ java -Djava.security.manager=allow -cp . -Djava.security.policy=server.policy servidor.LanzadorServidor 1100 Replica2 rmi://localhost:1099/Replica1
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by servidor.LanzadorServidor (file:/home/zapi24/Escritorio/3%202%c2%baCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src/)
WARNING: Please consider reporting this to the maintainers of servidor.LanzadorServidor
WARNING: System::setSecurityManager will be removed in a future release

=====
Replica2 ACTIVA y escuchando en puerto 1100
Conectada a 1 replicas compañeras.
=====
Cliente Paco registrado internamente.
Donación de 100.0 recibida de Paco

```

```

zapi24@zapi24-GF63-Thin-9SC:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P3/Practica3_Ejemplos/Practica3_Donaciones/src$ java -Djava.security.manager=allow -cp . -Djava.security.policy=cliente/client.policy cliente.Cliente
[OK] Conectado exitosamente a Replica1

--- MENU DE OPCIONES ---
Conectado actualmente a: Replica1
1. Registrarse como nuevo donante
2. Realizar una donacion
3. Consultar total recaudado (Global)
4. Ver lista de donantes (Global)
5. Salir
Elige una opcion: 2
Introduce tu ID de cliente: Zapi
Introduce la cantidad a donar: 50
>> ¡Gracias por tu donacion de 50.0 euros!

--- MENU DE OPCIONES ---
Conectado actualmente a: Replica1
1. Registrarse como nuevo donante
2. Realizar una donacion
3. Consultar total recaudado (Global)
4. Ver lista de donantes (Global)
5. Salir
Elige una opcion: 3
Introduce tu ID de cliente para verificar permisos: Zapi
>> TOTAL RECAUDADO GLOBALMENTE: 150.0 euros.

--- MENU DE OPCIONES ---
Conectado actualmente a: Replica1
1. Registrarse como nuevo donante
2. Realizar una donacion
3. Consultar total recaudado (Global)
4. Ver lista de donantes (Global)
5. Salir
Elige una opcion: 4
Introduce tu ID de cliente para verificar permisos: Zapi
>> LISTA DE DONANTES A NIVEL GLOBAL:
- Zapi
- Paco

```

Cabe recalcar antes de explicar la captura, que como la longitud de todos los pasos que hemos realizado en la tercera terminal es tan grande, no se ve el inicio de estos. Pero no pasa nada ya que si observamos las otras dos terminales, si podemos apreciar todo el proceso que se ha mencionado, como cada réplica ha recibido y almacenado su donación correspondiente de forma independiente.

Lo importante de esta captura se encuentra en la ejecución de las consultas. A pesar de que el cliente le está preguntando únicamente a la Réplica 1, el sistema nos devuelve un total de 150 euros y una lista que incluye tanto a Zapi como a Paco. Esto nos demuestra que la Réplica 1 suma su subtotal, realiza una petición a través de las funciones *getSubtotal()* y *getDonantesLocales()* a la Réplica 2, y fusiona ambos datos devolviendo la suma y la lista del sistema en tiempo real.

Dicho esto, queda verificado la robustez final del sistema, y el cumplimiento de todos los requisitos que se pedían en el guión para esta práctica.